



Instituto
de Ciencias
Tecnológicas



Framework de soluciones empresariales

Síntesis de la Unidad 3:

Acceso a Datos y Framework



Tema General

INTRODUCCIÓN

Prácticamente todas las aplicaciones de hoy en día requieren acceso al menos a una fuente de datos, dichas fuentes son por lo general base de datos relacionales, por lo que muchas veces no tenemos problema en acceder a los datos, sin embargo, hay ocasiones en las que necesitamos tener más de una fuente de datos o la fuente de datos que tenemos puede variar, lo que nos obligaría a modificar gran parte del código. Para esto, tenemos el patrón Arquitectónico Data Access Object (DAO), el cual permite separar la lógica de acceso a datos de los Bussines Objects u Objetos de negocios, de tal forma que el DAO encapsula toda la lógica de acceso de datos al resto de la aplicación.

1.Problemática

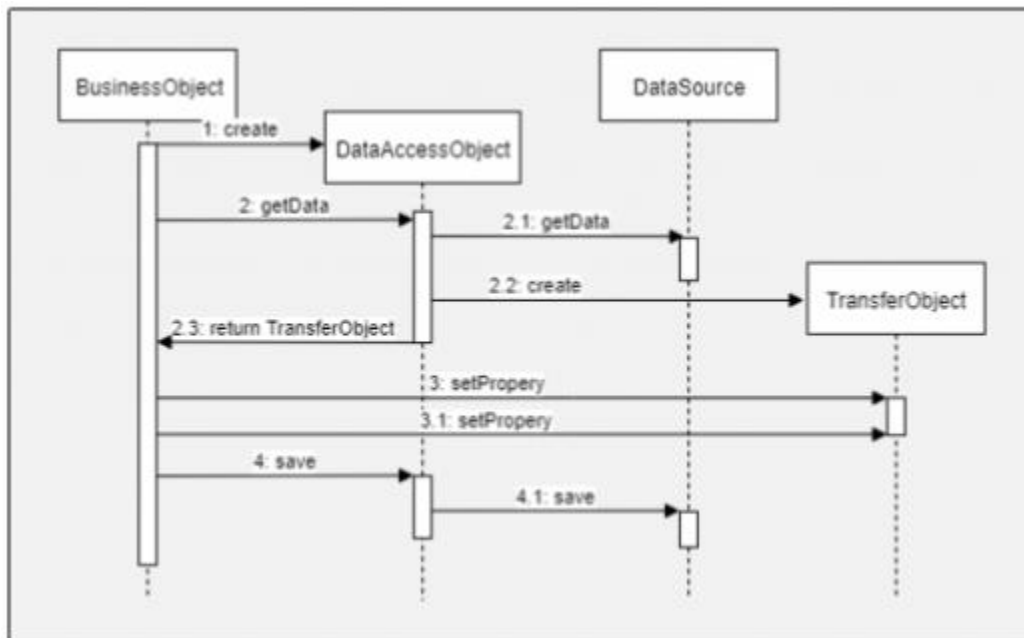
Uno de los problemas que nos encontramos al acceder a la fuente de datos, es que la implementación puede variar según la fuente de los datos. Implementar la lógica de acceso puede ser un problema, pues tendríamos que enfrentarnos a la lógica de negocio, más la lógica de acceso a datos, esto se torna mas complejo si tenemos múltiples fuentes de datos, teniendo como consecuencia múltiples lógicas para acceder a distintas fuentes de datos

2. DAO

Dado lo anterior, el patrón DAO propone separar por completo la lógica de negocio de la lógica para acceder a los datos, de esta forma, el DAO proporcionará los métodos necesarios para insertar, actualizar, borrar y consultar la información; por otra parte, la capa de negocio solo se preocupa por lógica de negocio y utiliza el DAO para interactuar con la fuente de datos.

Los compones que conforman el patrón son:

- **BusinessObject**: representa un objeto con la lógica de negocio.
- **DataAccessObject**: representa una capa de acceso a datos que oculta la fuente y los detalles técnicos para recuperar los datos.
- **TransferObject**: este es un objeto plano que implementa el patrón Data Transfer Object (DTO), el cual sirve para transmitir la información entre el DAO y el Business Service.
- **DataSource**: representa de forma abstracta la fuente de datos, la cual puede ser una base de datos, Webservices, LDAP, archivos de texto, etc.



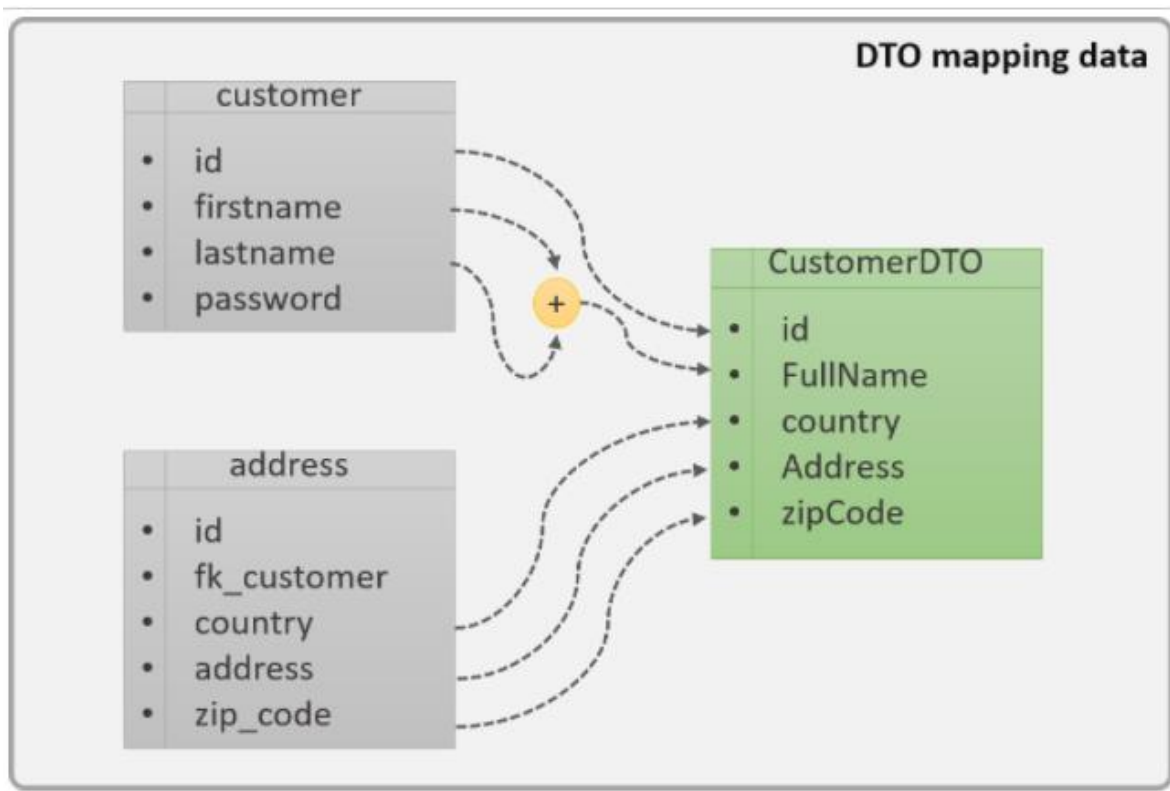
Fuente: <https://www.oscarblancarteblog.com/2018/12/10/data-access-object-dao-pattern/>

El diagrama se interpreta de la siguiente manera:

1. El *BusinessObject* crea u obtiene una referencia al *DataAccessObject*.
2. El *BusinessObject* solicita información al *DataAccessObject*
 - El *DataAccessObject* solicita la información al *DataSource*
 - El *DataAccessObject* crea una instancia del *TransferObject* con los datos recuperados del *DataSource*
 - El *DataAccessObject* response con el *TransferObject* creado en los pasos anteriores.
3. El *BusinessObject* actualiza algún valor del *TransferObject*
 - Más actualizaciones
4. El *BusinessObject* solicita el guardado de los datos actualizados al *DataAccessObject*.
 - El *DataAccessObject* guarda los datos en el *DataSource*.

3. DTO

Existe la problemática de querer recuperar datos de distintas tablas, para esta situación utilizamos el patrón DTO, Data Transfer Object, el cual tiene como finalidad crear un objeto plano (POJO), con una serie de atributos que puedan ser enviados o recuperados del servidor en una sola invocación, de tal forma que un DTO puede contener información de múltiples fuentes o tablas y concentrarlas en una única clase simple.



Fuente: <https://www.oscarblancarteblog.com/2018/11/30/data-transfer-object-dto-patron-diseno/>

En la imagen anterior podemos apreciar gráficamente como es que un DTO se conforma de una serie de atributos que puede o no, estar conformados por más de una fuente de datos. Para esto, el servidor obtiene la información de las tablas **customer** y **address** (izquierda) y realiza un mapping con el DTO (derecha). Adicional, la información puede ser pasada de un lado intacta como es el caso del **id**, **fullName**, **country**, **address** y **zipCode** o ser una derivada de más de un campo, como es el caso del **fullName**, el cual es la unión del **firstname** y **lastname**.

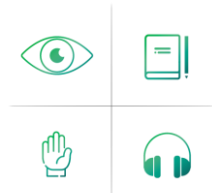
Otra de las ventajas no tan claras en la imagen, es que nos permite omitir información que el usuario no requiere, como es el caso de **password**. No es solo que no lo requiere, sino que además podría ser una gran falla de seguridad está enviando los **passwords**, es por ello que en el DTO lo omitimos.

5. Características de un DTO

Si bien un DTO es simplemente un objeto plano, sí que tiene que cumplir algunas reglas para poder considerar que hemos creado un DTO correctamente implementado:

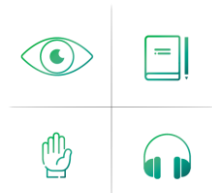
- **Solo lectura:** Dado que el objetivo de un DTO es utilizarlo como un objeto de transferencia entre el cliente y el servidor, es importante evitar tener operaciones de negocio o métodos que realicen cálculos sobre los datos, es por ello que solo deberemos de tener los métodos GET y SET de los respectivos atributos del DTO.
- **Serializable:** Es claro que, si los objetos tendrán que viajar por la red, deberán de poder ser serializables, pero no hablamos solamente de la clase en sí, sino que también todos los atributos que contenga el DTO deberán ser fácilmente serializables. Un error clásico en Java es, por ejemplo, crear atributos de tipo Date o Calendar para transmitir la fecha u hora, ya que estos no tienen una forma estándar para serializarse por ejemplo en Webservices o REST.

¿Qué soluciona DAO?:

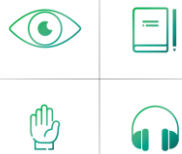


1. Problema al implementar la lógica de acceso a datos. (Correcta)
2. Problema al ejecutar los métodos relativos a un CRUD.
3. Problema al crear la conexión a una base de datos.

En resumen, que hace DAO:



1. Separa la lógica de negocio de la lógica para acceder a los datos (Correcta)
2. Separa las reglas del negocio de la lógica del controlador.
3. Separa las consultas de la lógica de acceso a datos.



¿Para qué sirve DTO?:

1. Para recuperar datos de distintas tablas (Correcta).
2. Para crear múltiples insertados en tablas de la base de datos.
3. Para crear un acceso a datos.

Conclusión

El patrón DAO es sin lugar a duda, uno de los más utilizados en la actualidad, ya que es fácil de implementar y proporciona claros beneficios, incluso, si solo tenemos una fuente de datos y esta no cambia, pues permite separar por completo la lógica de acceso a datos en una capa separada y así solo nos preocupamos por la lógica de negocio sin preocuparnos de donde viene los datos o los detalles técnicos para consultarlos o actualizarlos, por otro lado los DTO son un patrón muy efectivo para transmitir información entre un cliente y un servidor, pues permite crear estructuras de datos independientes de nuestro modelo de datos, lo que nos permite crear cuantas "vistas" sean necesarias de un conjunto de tablas u orígenes de datos.

Bibliografía

Referencia del presente documento:

<https://www.oscarblancarteblog.com/2018/12/10/data-access-object-dao-pattern/>

<https://www.oscarblancarteblog.com/2018/11/30/data-transfer-object-dto-patron-diseno/>